



RISC-V Trace Verification: Block Level CoCoTb based Verification in Shakthi C-Class Processor

PROJECT PHASE-I REPORT

Submitted by

SRINATH S (2127220701138)

VIJAI SARAVANAN R G (2127220701160)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SRI VENKATESWARA COLLEGE OF ENGINEERING

(An Autonomous Institution ; Affiliated to Anna University, Chennai -600 025)

ANNA UNIVERSITY:: CHENNAI 600 025

DECEMBER 2025

SRI VENKATESWARA COLLEGE OF ENGINEERING

(An Autonomous Institution ; Affiliated to Anna University, Chennai -600 025)

ANNA UNIVERSITY, CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project phase-I report **RISC-V Trace Verification: Block Level CoCoTb based Verification in Shakthi C-Class Processor** is the bonafide work of **SRINATH S (2127220701138)** and **VIJAI SARAVANAN R G (2127220701160)** who carried out the project work under my supervision.

SIGNATURE

Dr. N. KUMARATHARAN M.E., Ph.D.

PROFESSOR

HEAD OF THE DEPARTMENT

Department of Electronics and
Communication Engineering

SIGNATURE

Mr. SILAMBARASAN D M.E.

SUPERVISOR

ASSISTANT PROFESSOR

Department of Electronics and
Communication Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

RISC-V is an open-standard Instruction Set Architecture that enables flexible and modular processor design. Ensuring correct instruction execution is crucial for maintaining architectural compliance in RISC-V based processors. This project focuses on block-level trace verification of the Shakti C-Class RISC-V core to validate its instruction execution flow and identify deviations, if any, during runtime.

The verification methodology was implemented using a CoCoTb-based Python testbench integrated with the Verilator simulation environment. Assembly test programs were compiled into ELF files, disassembled for reference, and executed on the Shakti RTL model. Program Counter (PC) values captured during RTL simulation were logged and compared with the disassembly sequence to detect missing or misaligned instruction traces. Partial data verification was also carried out by comparing the final register values obtained from Spike with those from RTL execution for selected tests.

The developed trace verification flow effectively identified missing PC entries and supported systematic debugging of instruction flow inconsistencies. This work establishes a structured foundation for trace-based verification and can be extended towards full data trace validation, enhanced trace automation, and core-level verification in future work.

ACKNOWLEDGEMENT

We express our heartfelt thanks to the management of **SRI VENKATESWARA COLLEGE OF ENGINEERING** for providing us the platform for the project work. We sincerely thank our Principal **Dr. S. GANESH VAIDYANATHAN** for permitting us to carry out the project in the college.

We express our sincere thanks to the Head of the Department **Dr. N. KUMARATHARAN**, Department of Electronics and Communication Engineering, for the constant encouragement given to us for carrying out our project to gain practical knowledge in the field of study. We are grateful to **Mr. SILAMBARASAN D**, Assistant Professor, Supervisor, Department of Electronics and Communication Engineering, for his guidance and motivation during our project.

We would like to extend our sincere gratitude to our industry mentors from **VYOMA SYSTEMS** — **Ms. LAVANYA J** (CEO / Founder of Vyoma Systems | RISC-V Ambassador for India), **Mr. KUMARAN S** (Verification Engineer), and **Ms. RIZWANA G** (SaaS Lead) — for their invaluable technical support, guidance, and constant encouragement throughout the industrial project. Their insights in RISC-V verification and processor design greatly enhanced the depth and quality of our work.

We are grateful for the invaluable support provided by **Dr. D. MENAKA**, Associate Professor, **Mrs. R. KOUSALYA**, **Mrs. L. ANJU**, and **Mr. S. ELANGO VAN**, Assistant Professors, Department of Electronics and Communication Engineering, which played a significant role in the successful completion of our project.

We also wish to express our sincere thanks to all teaching and non-teaching staff members of the Department of Electronics and Communication Engineering and our friends for their kind help during the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 OVERVIEW OF RISC-V ARCHITECTURE	1
	1.2 SIGNIFICANCE OF RISC-V TRACE VERIFICATION	1
	1.3 OVERVIEW OF SHAKTI C-CLASS PROCESSOR	2
	1.4 SIGNIFICANCE OF TRACE VERIFICATION IN PROCESSOR	2
	1.5 SCOPE OF THE PROJECT	2
2	LITERATURE SURVEY	4
3	SYSTEM REQUIREMENTS AND METHODOLOGY	7
	3.1 SYSTEM REQUIREMENTS	7
	3.1.1 Hardware Requirements	7
	3.1.2 Software Requirements	7
	3.1.3 Functional Requirements	8
	3.2 METHODOLOGY	9
4	STUDY OF RISC-V ASSEMBLY AND TESTING	12
	4.1 RISC-V EXECUTION MODES	12

	4.2 STRUCTURE OF RISC-V ASSEMBLY LANGUAGE	14
	4.3 GENERATING ELF AND DISASSEMBLY FILES	14
	4.4 RUNNING ASSEMBLY TESTS IN SPIKE SIMULATOR	16
	4.5 UNDERSTANDING THE DISASSEMBLY OUTPUT	16
	4.6 DEBUGGING AND VALIDATION THROUGH ASSEMBLY-LEVEL TESTING	17
	4.7 IMPORTANCE OF ASSEMBLY TESTING IN VERIFICATION FLOW	18
5	COCOTB-BASED VERIFICATION FRAMEWORK	19
	5.1 OVERVIEW OF COCOTB FRAMEWORK	19
	5.2 NEED FOR COCOTB IN PROCESSOR VERIFICATION	20
	5.3 INTEGRATION WITH VERILATOR	21
	5.4 STRUCTURE OF COCOTB TESTBENCH	21
	5.5 VERIFICATION OF BASIC VERILOG DESIGNS	22
	5.6 ADVANTAGES OF COCOTB FRAMEWORK	23
6	RISC-V PROCESSOR TRACE ARCHITECTURE	24
	6.1 OVERVIEW OF RISC-V PROCESSOR TRACE	24
	6.2 TRACE SYSTEM ARCHITECTURE	25
	6.3 HART-TO-ENCODER INTERFACE	25
	6.4 TRACE ENCODER	26
	6.5 TRACE OUTPUT PACKETS	27
	6.6 TRACE DECODER	28

7	INSTRUCTION TRACE VERIFICATION	30
	7.1 OVERVIEW OF INSTRUCTION TRACE VERIFICATION PROCESS	30
	7.2 GENERATION OF ELF AND DISASSEMBLY FILES	31
	7.3 COCOTB-BASED TRACE LOGGING	31
	7.4 TRACE ANALYSIS AND IDENTIFICATION OF MISSING INSTRUCTIONS	32
	7.5 EXTRACTED MISSING INSTRUCTION REPORT	32
	7.6 DISASSEMBLED INSTRUCTION MAPPING	33
	7.7 INTERPRETATION OF TRACE RESULTS	33
8	DATA TRACE VERIFICATION	35
	8.1 OVERVIEW OF DATA TRACE MECHANISM	35
	8.2 METHODOLOGY FOR DATA TRACE VERIFICATION	36
	8.3 OBSERVATIONS FROM DATA TRACE RESULTS	37
9	SIMULATION AND RESULTS	38
	9.1 SIMULATION ENVIRONMENT SETUP	38
	9.2 COCOTB SIMULATION OUTPUT	39
	9.3 TRACE LOG AND RESULT FILES	40
	9.4 Analysis of Instruction Trace Results	41
	9.5 INTERPRETATION OF RESULTS	42
	9.6 SUMMARY OF MISSING INSTRUCTIONS - IMAFDC TEST SUITES	43
10	CONCLUSION AND FUTURE WORK	45
	REFERENCES	48

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
6.1	CoCoTb Signal and Hierarchical Mapping for Instruction Trace Access	29
7.1	Extracted Missing Instruction Report for ADDW Test	33
7.2	Disassembled Instruction Mapping for Missing Instructions	33
9.1	Consolidated Missing Instruction Summary for IMAFDC Suites	44

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Flowchart	11
6.1	RISC-V Trace Output Packet Format	27
9.1	Simulation Console Output Showing Logged PC Values During Instruction Trace Capture	39
9.2	Simulation Summary Showing Test Status and Termination	40
9.3	Files Generated for ADDW Instruction Trace Verification	41
9.4	Output of Trace Analysis Script Highlighting Missing PC Entries for the ADDW Test	42

LIST OF ABBREVIATIONS

S. NO.	ABBREVIATION	FULL FORM
1	ISA	Instruction Set Architecture
2	RTL	Register Transfer Level
3	ELF	Executable and Linkable Format
4	PC	Program Counter
5	CSV	Comma-Separated Values
6	CoCoTb	Coroutine-based Co-Simulation Testbench
7	DUT	Device Under Test
8	SoC	System on Chip
9	RISC-V	Reduced Instruction Set Computer – Five
10	GCC	GNU Compiler Collection

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW OF RISC-V ARCHITECTURE

RISC-V is an open-source Instruction Set Architecture (ISA) designed with simplicity, modularity, and extensibility in mind. It emerged as a response to the growing need for a free and standardized processor architecture that could be implemented and modified by both academia and industry without licensing restrictions. The RISC-V ISA follows the load-store principle, ensuring that all operations are performed through registers, thereby improving performance and reducing hardware complexity. It supports several base and extension instruction sets, including integer (I), multiplication (M), atomic (A), floating-point (F and D), and compressed (C) instruction sets, which can be selectively combined depending on the processor's application. Its open standard has made it a global foundation for both embedded and high-performance computing systems, enabling flexible customization and architectural experimentation.

1.2 SIGNIFICANCE OF RISC-V TRACE VERIFICATION

Verification is a vital phase in the hardware design cycle. In the case of RISC-V processors, which are often implemented by independent organizations, trace verification ensures that the processor faithfully executes every instruction as per the ISA specification. Trace verification involves capturing detailed information about executed instructions—specifically program counters (PCs), branching behaviour, and memory operations—during simulation or real-time execution. This captured information, called the trace, is then compared against reference data or expected outcomes to identify any discrepancies. The trace verification process validates pipeline operation, instruction sequencing, and control flow correctness. For the Shakti C-Class RISC-V processor, implementing trace verification through CoCoTb allows dynamic observation of the processor's internal behaviour. This approach ensures that

every instruction is properly executed, logged, and verified, improving the reliability and compliance of the design.

1.3 OVERVIEW OF SHAKTI C-CLASS PROCESSOR

The Shakti C-Class processor is a 64-bit RISC-V based in-order core designed at IIT Madras as part of the Shakti Project. It implements the RV64IM ISA subset and supports privileged modes defined in the RISC-V privileged specification. The C-Class core targets embedded and IoT applications and is optimized for low area and power consumption while maintaining full compliance with the RISC-V ecosystem. Its modular design, open-source RTL, and toolchain support make it an excellent candidate for academic-level verification and architectural research. The processor integrates interfaces for debugging, tracing, and external communication, which facilitates deeper hardware-software co-validation.

1.4 SIGNIFICANCE OF TRACE VERIFICATION IN PROCESSOR VALIDATION

In modern processor design, functional correctness cannot be guaranteed solely by running application-level programs. Even when final register states appear correct, there may be hidden issues such as pipeline hazards, incorrect instruction retirements, or mismanaged control flows. Trace verification plays an indispensable role in identifying such issues by providing a complete execution log. By tracing program counters, instruction types, and execution contexts, engineers can reconstruct the exact sequence of processor events. This ensures that the micro-architectural behaviour aligns with the expected instruction semantics. Moreover, trace verification enhances debuggability, as developers can pinpoint specific instruction cycles that deviate from expected behaviour.

1.5 SCOPE OF THE PROJECT

This project focuses on implementing instruction trace verification for the Shakti C-Class RISC-V processor using the CoCoTb framework. The objective is to

develop a block-level verification methodology that captures, analyzes, and validates the program counter traces generated during simulation. The work includes setting up the RISC-V environment, creating assembly test programs, generating ELF and disassembly files, and performing CoCoTb-based simulations integrated with Verilator. The captured traces are compared with the RTL dump to identify any mismatches. Partial data trace verification has also been demonstrated by comparing register states. The project does not include the complete integration of the golden reference model but serves as a foundation for advanced verification setups in future work.

CHAPTER 2

LITERATURE SURVEY

The development of a verification framework for a RISC-V processor requires a solid understanding of its architectural foundations, existing research on trace methodologies, and available verification infrastructures. This chapter surveys relevant technical literature that underpins the design and implementation of the present work.

The official RISC-V Instruction Set Manual [1] defines the unprivileged ISA, describing the base integer operations, load/store model, and control-flow instructions. It establishes the semantics of each instruction that hardware implementations must obey. The unprivileged specification is deliberately simple, enabling efficient hardware decoding and supporting both 32-bit and 64-bit architectures. Its open-source nature fosters academic experimentation and industrial adoption without licensing restrictions.

Complementing this is the privileged architecture specification [2], which extends the ISA to system-level functionality. It defines the execution modes—Machine, Supervisor, and User—along with trap, interrupt, and exception handling mechanisms. The specification also introduces Control and Status Registers (CSRs) that govern privilege transitions, interrupt enable bits, and virtual-memory translation. Together, the unprivileged and privileged volumes form the complete functional model that every compliant RISC-V processor must implement.

The Shakti Project documentation [3] describes the C-Class core developed at the Indian Institute of Technology Madras. The C-Class is a 64-bit in-order pipeline that implements the RV64IM ISA. It provides debug and trace interfaces that make it suitable for both industrial verification and educational use. The Shakti framework emphasizes openness, reproducibility, and modularity, aligning closely with the RISC-V design philosophy presented in [1] and [2].

Industrial verification practices for Shakti processors are further detailed by Vyoma Systems [4], which explains the adoption of open-source verification environments, Python-based scripting, and reusable testbench methodologies. Their framework demonstrates how block-level verification using tools such as CoCoTb can efficiently validate instruction and data trace correctness. This reference provides insight into applying modern co-simulation techniques to RISC-V designs.

The CoCoTb framework [5] is a coroutine-driven co-simulation library for verifying digital designs using Python. It enables direct interaction with HDL signals from a high-level language, offering the flexibility of software scripting within hardware verification. CoCoTb’s asynchronous testbench model significantly reduces verification complexity compared to purely HDL-based environments, allowing dynamic stimulus generation and result checking in real time. This methodology is central to the present project’s verification approach. The GNU RISC-V GCC Toolchain [6] provides the essential software environment for compiling and linking assembly programs into ELF binaries and generating disassembly files. This toolchain forms the bridge between software development and hardware simulation, ensuring that the generated test programs strictly conform to the RISC-V instruction semantics described in [1]. Its integration into automated verification workflows is standard practice across both academic and industrial domains.

Accurate simulation of RISC-V hardware requires adherence to the SystemVerilog Language Reference Manual [7]. The IEEE 1800-2017 standard defines syntax and semantics for designing and verifying RTL hardware. Verilator, the simulation backend used in this work, follows these standards while translating SystemVerilog constructs into C++ models compatible with CoCoTb. The standard also establishes best practices for assertions, coverage, and modular hierarchy representation, all of which enhance trace observability.

Foundational concepts in computer organization are drawn from Patterson and Hennessy's classic textbook [8], which elaborates on pipeline hazards, control flow, and data-path design. These principles directly influence how instruction traces are interpreted during verification. Similarly, The RISC-V Reader by Patterson and Waterman [9] offers a concise yet comprehensive explanation of the RISC-V ISA, serving as a reference for interpreting the operational meaning of each instruction captured in the trace logs.

Collectively, these references establish the theoretical basis for implementing and validating a RISC-V trace verification environment. The unprivileged and privileged specifications [1][2] define what must be verified, the Shakti and Vyoma documentation [3][4] explain how verification is integrated in practice, and the CoCoTb and SystemVerilog standards [5][7] describe the tools that make such verification possible. The academic texts [8][9] provide the architectural understanding required to interpret the captured instruction behaviour. Together, they form the comprehensive literature foundation for this project's methodology and implementation.

CHAPTER 3

SYSTEM REQUIREMENTS AND METHODOLOGY

This chapter outlines the hardware and software requirements essential for implementing the RISC-V trace verification environment, followed by a detailed explanation of the adopted methodology. The methodology describes how the verification flow was structured to ensure reliable instruction trace validation using the CoCoTb framework in conjunction with Verilator. The experimental framework aims to validate the program counter trace of the Shakti C-Class RISC-V processor through systematic simulation and analysis.

3.1 SYSTEM REQUIREMENTS

3.1.1 Hardware Requirements

The implementation of trace verification involves resource-intensive simulation and waveform generation processes, which demand a high-performance workstation. The hardware platform used in this project consisted of a multi-core Intel i5 processor operating at a base frequency of 3.0 GHz, supported by 8 GB of DDR4 RAM and a minimum of 50 GB of free storage space. These specifications were essential to handle large simulation traces and CoCoTb testbench logs efficiently. The machine was configured with a 64-bit operating system to support the RISC-V cross-compilation toolchain and Verilator build processes. A higher-end processor with additional cores or threads can further enhance simulation speed, but the chosen configuration proved sufficient for this project's verification workloads.

3.1.2 Software Requirements

The verification setup was implemented on Ubuntu 20.04 LTS, chosen for its stability and compatibility with open-source design and verification tools. The RISC-V GCC toolchain was used to compile assembly source programs into ELF

binaries and generate corresponding disassembly files. These outputs served as the primary input stimuli for both Spike (the instruction set simulator) and the RTL-based simulation environment.

The Verilator tool was utilized to compile the Verilog RTL of the Shakti C-Class processor into an executable C++ model, providing a fast and accurate simulation of the design at the cycle level. Verilator is a widely adopted open-source simulator that offers near-commercial performance levels and supports integration with external testbenches.

The CoCoTb (Coroutine-based Co-simulation Testbench) framework, developed in Python, served as the verification driver for the simulation environment. CoCoTb provides a flexible interface between high-level scripting and low-level hardware signals, enabling real-time signal access, logging, and result comparison. Python's asynchronous coroutine structure allowed efficient handling of concurrent simulation tasks, making it possible to log program counters and automate comparison scripts without requiring custom HDL testbenches.

Additionally, Surfer was used for waveform visualization, GNU Make for build automation, and Python's Pandas and CSV libraries for log parsing and data comparison. These tools collectively provided a robust environment for executing and analyzing the verification results.

3.1.3 Functional Requirements

To achieve accurate instruction trace verification, the system had to fulfill several functional requirements. The verification framework required full compatibility between the RISC-V GCC toolchain, the Verilator simulation output, and the CoCoTb testbench interface. The environment had to automatically execute assembly-level test programs, generate ELF and disassembly files, and run Spike simulations to establish reference correctness. During CoCoTb simulation, the

system needed to record program counter (PC) values dynamically from the signal

dut.soc.soc.ccore_0.riscv_etrace_ingress_port_iaddr.value

which serves as the instruction address trace port in the Shakti C-Class processor.

After capturing the PC stream, the verification scripts were responsible for comparing the recorded PC sequence against the reference RTL dump file generated by the simulation. Any PCs that were missing or misaligned were extracted and cross-referenced with the disassembly file to identify the corresponding instruction. Finally, the system was required to produce structured output reports in CSV and text formats, summarizing all observed discrepancies across different instruction test suites. This functional integration ensured a smooth and automated verification pipeline from input assembly to trace analysis.

3.2 METHODOLOGY

The methodology for instruction trace verification was developed in a phased manner, beginning with software-level verification and extending to block-level hardware verification using CoCoTb. The process was carefully designed to ensure that all test stages complemented one another and validated different aspects of the processor's behaviour.

The first stage involved a thorough study of the RISC-V unprivileged and privileged specifications to understand the expected instruction behaviour across different execution modes. This theoretical foundation was essential for creating relevant test cases that could effectively verify both user-level and system-level operations of the Shakti C-Class processor.

The next step focused on assembly-level testing. A set of assembly test programs was created for basic arithmetic and control operations, including the addw instruction. These test programs were compiled using the RISC-V GCC toolchain

to produce ELF binaries and disassembly files. The ELF file represented the executable code loaded into the processor memory, while the disassembly file provided a human-readable interpretation of the same. Each ELF file was executed in the Spike ISA simulator, which served as a reference model to confirm the correctness of instruction behaviour at the architectural level. Successful Spike execution indicated that the test code itself was valid and suitable for RTL-level verification.

After verifying functionality at the software level, the process moved to hardware-level simulation. The Verilog RTL of the Shakti C-Class processor was compiled using Verilator to generate a C++ simulation model. This model was then interfaced with the CoCoTb Python testbench. The CoCoTb environment was responsible for initializing the simulation, applying input stimuli, monitoring internal signals, and collecting the program counter (PC) trace values. These logged PCs represented the instruction addresses corresponding to each execution cycle. The CoCoTb testbench was designed to terminate the simulation automatically using the `$finish` statement once all instructions had been executed, marking the end of the test sequence.

The recorded PC values were post-processed using a Python script that compared them with the RTL dump file generated during simulation. Any program counter values present in the RTL dump but missing from the CoCoTb logs were flagged as missing or untraced instructions. To identify the corresponding assembly operation, these PCs were cross-referenced with the disassembly file. This automated process ensured accurate identification of missing instructions and made it possible to generate comprehensive trace coverage reports.

The entire methodology was repeated for different test suites, such as rv64ui, rv64ua, and rv64uc, in both p-mode (privileged mode) and v-mode (virtualized mode). The resulting data was analyzed to determine whether instruction flow and privilege transitions occurred correctly during execution. This step-by-step

methodology ensured complete verification coverage from the instruction level to the architectural behaviour of the processor.

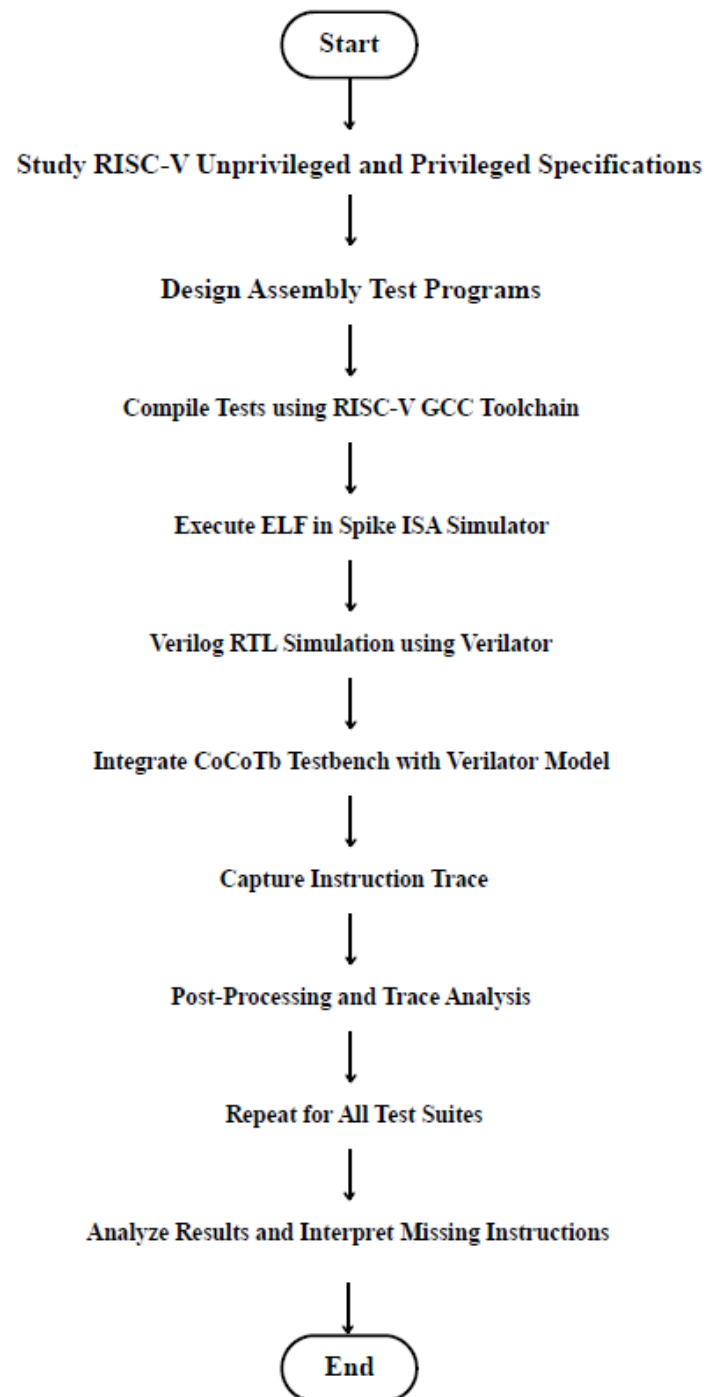


Fig 3.1 Flowchart

CHAPTER 4

STUDY OF RISC-V ASSEMBLY AND TESTING

The functional verification of a processor begins at the assembly level, where each instruction can be evaluated for correctness in isolation. Studying RISC-V assembly language provides an understanding of how individual instructions are encoded, executed, and influence processor state transitions. Before initiating block-level or trace-based verification, it is essential to ensure that the assembly-level tests used as inputs are syntactically correct and semantically valid. This chapter discusses the structure of RISC-V assembly programs, the generation of essential test files such as ELF and disassembly outputs, and the use of the Spike simulator to establish reference correctness for these programs.

4.1 RISC-V EXECUTION MODES

The RISC-V architecture defines multiple privilege levels that control access to hardware resources and determine the scope of executable operations. These modes ensure isolation between user applications and system control mechanisms. The Shakti C-Class processor implements the three standard RISC-V privilege levels: User Mode, Supervisor Mode, and Machine Mode.

User Mode (U-Mode)

User Mode is the least privileged execution environment, intended primarily for running user applications. In this mode, the processor can execute general-purpose instructions but is restricted from accessing system control registers or performing operations that could affect global system state. Attempts to execute privileged instructions in this mode trigger traps that transfer control to a higher privilege level, typically Supervisor or Machine mode. The User Mode ensures isolation between user processes and kernel operations, maintaining system stability and security. In trace verification, instruction traces from User Mode are

essential for validating that application-level programs execute correctly without privilege violations.

Supervisor Mode (S-Mode)

Supervisor Mode serves as the intermediary between application programs and the hardware. It is primarily responsible for managing system software tasks such as memory management, interrupt handling, and input/output control. This mode has access to page tables and can manage virtual-to-physical address translation. Supervisor Mode can also delegate certain traps or exceptions to Machine Mode when higher-level intervention is required. The Shakti C-Class processor, in compliance with the RISC-V privileged specification, uses Supervisor Mode for executing operating system kernels and device drivers. Trace verification in this mode ensures that all transitions between user and supervisor levels occur seamlessly and that virtual memory and system calls are correctly managed.

Machine Mode (M-Mode)

Machine Mode is the most privileged level in the RISC-V hierarchy. It has complete access to all hardware resources, including configuration registers, interrupt enable bits, and trap vectors. The processor begins execution in Machine Mode after reset, initializing memory and system peripherals before delegating control to lower privilege levels. It also manages exception delegation and interrupt handling for all modes. In the Shakti C-Class processor, the Machine Mode serves as the boot and trap handler level. Trace verification in this mode validates that initialization routines and privileged operations, such as CSR writes (csw) and returns from exceptions (mret), are performed according to the RISC-V privileged architecture specification. Correct operation in this mode ensures reliable system startup and stable hardware control.

4.2 STRUCTURE OF RISC-V ASSEMBLY LANGUAGE

The RISC-V assembly language follows a simple and consistent syntax, reflecting the clean design philosophy of the ISA itself. Each instruction consists of an operation mnemonic followed by destination and source operands. For example, the instruction `add x5, x6, x7` adds the contents of registers `x6` and `x7` and stores the result in register `x5`. The architecture adopts a fixed 32-bit instruction length for the base ISA, which simplifies decoding and execution logic within the processor pipeline.

RISC-V assembly supports three fundamental instruction types: arithmetic and logical instructions, control-flow instructions, and memory-access instructions. Arithmetic and logical operations manipulate data stored in registers, while control-flow instructions alter the program counter (PC) to implement branching, jumps, and function calls. Memory-access instructions are responsible for transferring data between registers and memory.

Each instruction executes in a single or few pipeline cycles, depending on the core's micro-architecture. For verification purposes, these low-level operations provide the most transparent representation of processor behaviour. Consequently, analyzing instruction execution at the assembly level ensures that subsequent stages of trace verification capture correct and expected operations.

4.3 GENERATING ELF AND DISASSEMBLY FILES

In order to test a RISC-V processor design, assembly code must be compiled into machine-readable form. This is achieved by using the RISC-V GCC toolchain, which converts assembly source files into ELF (Executable and Linkable Format) binaries. The ELF format is a standard executable representation used by most modern architectures. It contains all relevant sections, including text (instructions), data, and symbol tables.

For this project, the ELF file was generated using the following compilation command:

```
riscv64-unknown-elf-gcc -march=rv64g -mabi=lp64 -static -  
mcmmodel=medany -nostdlib -nostartfiles -fvisibility=hidden -  
I./env -I./env/p -I./isa/macros -I./isa/macros/scalar -  
T./env/p/link.ld addw.S -o addw.elf
```

This command compiles the assembly source `addw.S` into the executable `addw.elf` using the RV64G instruction set. The `-march` and `-mabi` flags specify the target architecture and calling convention, respectively. The `-nostdlib` and `-nostartfiles` options remove standard startup files and libraries to ensure that only user-defined code is compiled, as required for bare-metal testing. The linker script (`link.ld`) defines the memory layout for code and data segments.

After generating the ELF file, it is necessary to convert it into a human-readable form to analyze the corresponding instruction sequence. This is done by creating a disassembly file, which lists the memory addresses of each instruction along with their opcodes and mnemonics. The disassembly file for the above ELF binary is produced using the following command:

```
riscv64-unknown-elf-objdump -d addw.elf > addw.disass
```

The disassembly file serves as a crucial reference during verification. It enables a direct comparison between expected and observed instruction sequences. Each instruction in the disassembly is associated with its memory address, allowing verification scripts to correlate missing program counters (PCs) with specific instructions. During instruction trace verification, this disassembly output becomes the foundation for analyzing any missing or unmatched PCs recorded during simulation.

4.4 RUNNING ASSEMBLY TESTS IN SPIKE SIMULATOR

Before integrating the generated ELF file into the hardware verification setup, it is essential to validate its correctness at the instruction set level. This validation is carried out using Spike, the official RISC-V Instruction Set Simulator (ISS). Spike executes the ELF file in a software model of the RISC-V architecture, ensuring that each instruction behaves as defined in the ISA specification.

Running the ELF file in Spike provides two primary advantages. First, it confirms that the test program executes without illegal instruction exceptions or misaligned memory accesses. Second, it establishes a golden reference output, including the final register and memory states after program completion. These results can later be compared with the hardware simulation outputs to detect deviations in processor behaviour.

In this project, the Spike simulator was used to execute all test programs within the rv64ui and rv64ua suites. Each ELF file produced a set of register dumps and execution logs. The correct execution of these tests in Spike provided confidence that the test programs were valid and that any discrepancies observed later during RTL simulation were attributable to design or trace-level issues rather than to the test code itself.

4.5 UNDERSTANDING THE DISASSEMBLY OUTPUT

The disassembly file generated earlier serves as the textual interpretation of the machine code contained in the ELF binary. Each line in the file consists of three components: the instruction address (or PC value), the hexadecimal opcode, and the corresponding assembly mnemonic.

Here, the address 0x800000B6 indicates the memory location of the instruction `csw 0x744,8`, while the address 0x80000062 corresponds to the store instruction `sw t5,12(t6)`. During instruction trace verification, these memory addresses become the key identifiers used to verify the presence or absence of

corresponding program counter (PC) entries in the trace log. The CoCoTb simulation records a continuous stream of PCs as the processor executes each instruction. By cross-referencing these PCs with the disassembly file, it becomes possible to identify exactly which instructions were executed, skipped, or not captured in the trace.

Table 4.1 Missing Instruction Disassembly for ADDW Test

Instruction Address (PC)	Machine Code (Hex)	Assembly Instruction
0x800000B6	00008773	csrw 0x744,8
0x80000062	00C2A023	sw t5,12(t6)

The disassembly output is also used to analyze branching and jump behaviour. Since branch instructions modify the PC non-sequentially, verifying their corresponding entries in the trace ensures that control-flow operations are accurately logged. This makes the disassembly an indispensable element of the verification process.

4.6 DEBUGGING AND VALIDATION THROUGH ASSEMBLY-LEVEL TESTING

Assembly-level testing allows fine-grained observation of processor operations. Unlike high-level programming, assembly offers precise control over the sequence and type of instructions executed. This level of transparency makes it easier to identify and isolate hardware-level issues. For instance, if an instruction behaves incorrectly or is missing in the trace logs, its exact mnemonic and address can be identified directly from the disassembly file.

In this project, assembly-level debugging was used to verify pipeline synchronization, correct register updates, and proper termination of test programs. The outputs from the Spike simulator and disassembly analysis were combined to ensure that both the expected and actual results matched perfectly at

the instruction level. Any mismatch detected here would point to design or simulation issues before moving to trace-based validation.

4.7 IMPORTANCE OF ASSEMBLY TESTING IN VERIFICATION FLOW

The entire verification process builds upon the foundation of assembly-level correctness. Before performing instruction trace verification using CoCoTb, it is critical to ensure that each assembly test executes correctly in the software simulation environment. This step guarantees that any anomaly detected during the hardware-level verification originates from the RTL implementation rather than from errors in test generation.

Assembly testing establishes baseline confidence in both the instruction flow and functional behaviour of the design. Once verified, the same ELF and disassembly files can be safely used in CoCoTb simulations, allowing deeper analysis of program counters and instruction traces. Therefore, the assembly testing stage serves as the golden reference layer for the entire trace verification process, bridging the gap between theoretical ISA correctness and practical RTL-level validation.

CHAPTER 5

COCOTB-BASED VERIFICATION FRAMEWORK

The traditional approach to hardware verification primarily relies on HDL-based testbenches written in Verilog or SystemVerilog. While these methods are effective for structural verification, they often lack flexibility and automation when used in large-scale or iterative testing environments. The growing complexity of modern System-on-Chip (SoC) designs demands a more versatile framework that can seamlessly interface with simulation tools, automate data analysis, and provide rapid prototyping capabilities. The CoCoTb (Coroutine-based Co-simulation Testbench) framework fulfills these requirements by combining hardware simulation with the flexibility of the Python programming language.

This chapter provides an overview of the CoCoTb framework, its integration with Verilator for RISC-V processor verification, and the specific role it played in validating instruction trace behaviour in the Shakti C-Class processor.

5.1 OVERVIEW OF COCOTB FRAMEWORK

CoCoTb is an open-source co-simulation framework that enables verification of hardware designs using Python-based testbenches. It provides a bridge between Python scripts and HDL simulators by leveraging the Foreign Language Interface (FLI), VPI (Verilog Procedural Interface), or VHPI (VHDL Procedural Interface). This architecture allows Python code to control, monitor, and react to signal activity within a Verilog or VHDL simulation.

Unlike traditional HDL-based testbenches, CoCoTb separates the test logic from the design, allowing engineers to use Python's extensive libraries for file handling, data visualization, and statistical analysis. The testbench can dynamically drive signals, wait for specific events, read internal registers, and validate outputs—all from Python coroutines. CoCoTb also supports multi-

language simulation environments, making it suitable for verifying both FPGA-based and ASIC-based designs.

In the context of this project, CoCoTb was used to verify the instruction trace of the Shakti C-Class RISC-V processor by interacting with the Verilator-compiled model. Its asynchronous event-driven model made it ideal for capturing program counter updates and validating them against the reference trace data.

5.2 NEED FOR COCOTB IN PROCESSOR VERIFICATION

Processor verification typically requires an extensive set of test cases covering arithmetic operations, control instructions, exceptions, and privileged behaviours. HDL-based verification frameworks require lengthy compilation and waveform analysis cycles for each test. By contrast, CoCoTb provides rapid test execution with reusable Python scripts that can adapt to various configurations.

The choice of CoCoTb was particularly advantageous for the instruction trace verification phase of this project because the verification process involved large-scale log handling and comparison between multiple data sources (RTL dump, disassembly file, and logged PC trace). Python's ability to handle file input/output operations efficiently made the verification process both faster and more robust. CoCoTb's coroutine mechanism allowed asynchronous sampling of the instruction address signal without interrupting the simulation clock, ensuring that every executed instruction was recorded in real time.

Furthermore, the integration of CoCoTb with Verilator provided direct access to internal signals of the Shakti processor without modifying the RTL design. This made it possible to capture low-level events, such as program counter increments and CSR writes, which are otherwise inaccessible in higher-level verification frameworks.

5.3 INTEGRATION WITH VERILATOR

Verilator is an open-source cycle-accurate simulator that converts Verilog RTL into C++ models. This compiled model executes faster than traditional interpreted simulators and provides access to all signal hierarchies within the design. For this project, the Shakti C-Class processor's RTL files were compiled into a Verilator model, with the top-level module (Vtop) acting as the simulation root.

The CoCoTb testbench was interfaced with this Verilator model using the Verilog Procedural Interface (VPI). This interface allowed the Python testbench to read and write internal signal values dynamically during simulation. The CoCoTb environment was configured to monitor the signal path:

`dut.soc.soc.ccore_0.riscv_etrace_ingress_port_iaddr.value`

This signal represents the instruction address, or program counter (PC), at the ingress point of the trace encoder within the Shakti SoC. Capturing the real-time value of this signal was critical to reconstructing the instruction execution flow. The CoCoTb testbench continuously sampled this signal during simulation, storing each captured PC value into a structured log file for post-processing.

The simulation environment also supported automated resets, clock generation, and synchronization events. When the processor completed execution, the \$finish command was triggered within the Verilator environment, signaling CoCoTb to terminate the test gracefully. This design ensured that the test could stop automatically once the instruction stream had completed without manual intervention.

5.4 STRUCTURE OF COCOTB TESTBENCH

The CoCoTb testbench for this project consisted of several key components written in Python. The test initialization block loaded environment variables, parsed configuration parameters, and invoked the Verilator-compiled simulation

model. The clock coroutine was responsible for generating and controlling the simulation clock, ensuring proper timing across modules.

The monitor coroutine captured signals of interest, such as the instruction address, at every rising clock edge. Each captured value was appended to a Python list, which was later written to a CSV file for trace comparison. Additional coroutines managed reset behaviour, test completion, and result verification.

Each simulation run produced multiple files, including:

- `pc_log_<testname>.csv` – containing the list of all recorded program counter values.
- `rtl.dump` – representing the golden trace reference generated by Verilator.

By modularizing these tasks, CoCoTb simplified test management and enabled automated regression testing across multiple RISC-V suites.

5.5 VERIFICATION OF BASIC VERILOG DESIGNS

Before applying CoCoTb to the Shakti C-Class processor, preliminary tests were conducted on simpler Verilog modules such as adders, counters, and multiplexers. These small-scale designs served as training examples to understand CoCoTb's event-driven behaviour. By writing Python test scripts that applied input vectors, observed outputs, and checked expected results, the functionality of CoCoTb was validated.

For instance, an adder module was tested by generating random pairs of input values, applying them to the design, and asserting that the output matched the expected sum. A counter module was tested for correct increment and reset behaviour. These experiments helped establish confidence in the CoCoTb testbench structure and demonstrated its ability to perform synchronized read/write operations on HDL signals.

The successful completion of these tests provided a strong foundation for scaling the framework to a complex processor design. Once the environment was validated with basic modules, it was extended to interact with the Verilator-generated Shakti C-Class model to perform instruction trace verification.

5.6 ADVANTAGES OF COCOTB FRAMEWORK

CoCoTb provides several advantages over conventional HDL-based testbenches. The most significant advantage lies in its high-level scripting capability. Python’s dynamic nature allows engineers to modify tests, change parameters, or add new functionality without recompiling the HDL design. This makes it ideal for exploratory verification processes such as trace validation, where frequent test case updates are common.

Another key advantage is automation and scalability. The same CoCoTb script can be reused to verify multiple designs simply by changing configuration parameters. This flexibility dramatically reduces verification time when testing large numbers of RISC-V assembly programs.

Furthermore, CoCoTb supports direct integration with modern data analysis tools, enabling automated comparison of simulation results with reference outputs. In this project, Python scripts were used to automatically compare logged PCs with RTL dumps, identify missing addresses, and extract corresponding instructions from disassembly files. This seamless integration between verification and analysis made CoCoTb an indispensable part of the trace verification pipeline.

Finally, CoCoTb’s asynchronous coroutine model allows multiple processes—such as signal monitoring, event triggering, and file writing—to occur concurrently within a single simulation. This makes it possible to capture detailed instruction traces in real time without causing simulation slowdowns.

CHAPTER 6

RISC-V PROCESSOR TRACE ARCHITECTURE

Verification at the instruction-trace level requires a clear understanding of how trace information is generated, transmitted, and decoded within the processor. The RISC-V Processor Trace Architecture, as defined by the RISC-V Foundation, provides a standardized mechanism for capturing control-flow and event information from an executing core. Trace data primarily consists of changes in the program counter, branch events, and other control signals that describe the processor’s dynamic behaviour.

This chapter presents an overview of the trace system, explains the components involved in trace generation—including the trace encoder, output packet structure, and decoder—and describes how these concepts are applied in the verification of the Shakti C-Class processor.

6.1 OVERVIEW OF RISC-V PROCESSOR TRACE

The purpose of the RISC-V Processor Trace Specification is to enable external tools to reconstruct the exact sequence of executed instructions using a compressed and standardized trace stream. Instead of recording every instruction explicitly, the trace encoder captures only changes in the program counter (PC) and other events such as taken branches, exceptions, and interrupts. This approach significantly reduces data bandwidth while preserving the ability to reproduce program execution deterministically.

In a verification context, instruction trace analysis validates whether the captured sequence of PC values matches the architecturally expected order of execution. If the trace accurately reflects the processor’s behaviour, it can be used to prove functional correctness, debug runtime issues, and measure coverage. The Shakti C-Class processor implements trace functionality at the block level, where the

ingress port captures PC updates and passes them to a simulation-level encoder for verification.

6.2 TRACE SYSTEM ARCHITECTURE

A typical RISC-V trace subsystem comprises three major components:

1. Trace Source (Hart Interface) – Located inside the core, this unit provides information about instruction retirement and branch outcomes.
2. Trace Encoder – This component compresses event data into a structured packet format defined by the specification.
3. Trace Decoder – Used by analysis tools to reconstruct the instruction stream from the encoded packets.

Within the Shakti C-Class design, the trace source corresponds to the core's instruction-retirement logic, while the encoder and decoder are modeled at the simulation level. The trace encoder is connected through a hierarchical signal path that provides the current instruction address and other relevant metadata.

The encoder collects updates every time an instruction is retired or when a branch or exception changes control flow. These events are formatted into packets containing synchronization markers and compressed PC deltas. The packets can then be exported to external trace analyzers or, in this project's case, compared with the RTL dump for validation.

6.3 HART-TO-ENCODER INTERFACE

The interface between the RISC-V hart and the trace encoder is crucial because it defines how real-time information about instruction execution is transferred for tracing. This interface carries multiple signal types, including:

- Instruction Address (PC): Provides the address of the retired instruction.

- Branch Event Indicators: Identify whether the current instruction causes a branch, jump, or exception.
- Instruction Valid Signal: Indicates that the instruction has completed the pipeline and is ready for trace recording.

In the Shakti C-Class processor, this information is accessible through the Verilog signal path

`dut.soc.soc.ccore_0.riscv_etrace_ingress_port_iaddr.value`

which corresponds to the instruction address ingress port of the trace module.

During CoCoTb simulation, this signal is continuously monitored to record the PC value for each executed instruction.

The data captured from this interface forms the foundation for the instruction-trace verification flow. By comparing the recorded addresses with those in the RTL dump, the verification process can identify missing or incorrect instructions.

6.4 TRACE ENCODER

The trace encoder converts raw event data from the hart interface into compact trace packets. Instead of transmitting complete instruction information, the encoder records deltas—differences between successive PC values—and encodes special control-flow events such as jumps or traps. This design drastically reduces the data size of the trace stream.

In a hardware implementation, the encoder consists of combinational logic and finite-state machines that detect instruction retirement events, compress data fields, and output standardized trace packets. In this project’s simulation environment, the encoder’s conceptual function is emulated through CoCoTb scripts that log the instruction addresses at every cycle. Although a full hardware

encoder was not implemented, the recorded PC sequence effectively represents the same data that a hardware encoder would generate.

6.5 TRACE OUTPUT PACKETS

The RISC-V trace architecture uses a structured packet format to represent instruction trace information efficiently. Each trace packet carries essential data about instruction execution, program counter updates, and timing details. The packets are designed to minimize bandwidth while preserving complete information needed for reconstructing the instruction flow during analysis.

As shown in Figure 6.1, a typical trace packet consists of four primary fields: Header, Index, Optional Time Lag, and Payload.

The Header field (1 byte) defines the type of packet and its control attributes. The Index field, which varies in width depending on the implementation, identifies the source or context of the trace event. The Optional Time Lag field (2 bytes when present) specifies the delay or number of cycles since the last trace update, allowing timing correlation between events. The Payload field (1–31 bytes) contains the actual trace data, such as instruction address deltas or control-flow information.



Fig. 6.1 RISC-V trace output packet format

In this project's simulation-based environment, these hardware packets are represented in a simplified software form through structured logging. Each captured instruction address during CoCoTb simulation is recorded sequentially into the `pc_log_<test>.csv` file, which serves as the software equivalent of a trace

packet stream. These logs are later analyzed to verify whether the sequence of executed instructions aligns with the expected RTL behaviour.

This packet-based structure ensures a compact and organized way to represent instruction traces, enabling effective verification and debugging of processor execution flow.

6.6 TRACE DECODER

The trace decoder is responsible for reconstructing the original instruction sequence from the encoded trace packets. It interprets synchronization markers and PC deltas to regenerate the full sequence of instruction addresses. During hardware debugging, the decoder operates as part of an external trace tool that interfaces with the processor via JTAG or UART.

In this project, the trace decoder is functionally represented by Python scripts that compare logged PC values with those extracted from the RTL dump. Any mismatch indicates a missing or mis-sequenced instruction. The decoder also identifies start and stop synchronization points corresponding to processor initialization and end of execution.

Through this mechanism, the verification framework effectively validates both the completeness and accuracy of the captured instruction trace. The process ensures that every instruction executed within the core corresponds to a traceable PC entry and that the program counter progression conforms to the expected control-flow behaviour.

The hierarchical mapping between CoCoTb objects and Verilog signal paths used for extracting instruction trace information is summarized in Table 6.1.

Table 6.1 – CoCoTb Signal and Hierarchical Mapping for Instruction Trace Access

Term	Type	Role
dut	CoCoTb top-level handle	Acts as the simulation root object within the CoCoTb environment, representing the top of the Verilator hierarchy.
soc	Verilog submodule	The System-on-Chip wrapper that contains peripheral and core instances used for simulation.
ccore_0	Instance	Represents CPU core 0 within the SoC design. The C-Class is instantiated here as the primary processing unit.
riscv_etrace_ingress_port_iaddr	Signal	The instruction address ingress port used to provide real-time PC values to the trace encoder.
.value	CoCoTb object	Python-accessible attribute allowing read/write operations on the current HDL signal value during simulation.

CHAPTER 7

INSTRUCTION TRACE VERIFICATION

Instruction trace verification plays a crucial role in validating the correctness of program execution within a processor. While assembly-level testing and ISA simulation using Spike ensure expected behaviour at the software level, block-level verification through trace analysis confirms that the RTL implementation faithfully executes each instruction. This chapter presents the methodology followed for instruction trace validation of the Shakti C-Class processor using CoCoTb and Verilator, along with detailed analysis of the trace results.

The objective of instruction trace verification is to record the sequence of Program Counter (PC) values generated during execution and ensure that they match the reference trace produced by the RTL dump. Any missing or unmatched PCs indicate potential issues in trace capture, instruction retirement, or encoder visibility. The verification process was carried out for various instructions in the IMAFDC tests, and one representative test—the ADDW instruction—is explained in detail in this chapter.

7.1 OVERVIEW OF INSTRUCTION TRACE VERIFICATION PROCESS

The instruction trace verification process begins with the preparation of assembly-level test programs. The .S source file corresponding to the instruction under test is compiled using the RISC-V GCC toolchain to generate the ELF binary. The ELF file is then disassembled to produce the .disass file, which provides the reference instruction sequences along with their corresponding memory addresses. This disassembly acts as the lookup reference during the trace comparison stage.

The compiled ELF file is executed on the Verilator-simulated Shakti C-Class processor using CoCoTb. During simulation, the PC value is continuously sampled through the hierarchical trace port signal, enabling real-time monitoring

of instruction flow. These recorded PCs are stored in a CSV file. In parallel, Verilator generates an RTL dump file (rtl.dump) that lists the retired PCs. A Python-based analysis script compares the logged PCs with the RTL dump values to identify any missing or misaligned PC entries. The missing PC values are further mapped to their instructions using the disassembly file, enabling identification of instructions not captured in the recorded trace.

This systematic approach ensures that instruction traces are captured comprehensively and that discrepancies are traced to their corresponding instruction-level events.

7.2 GENERATION OF ELF AND DISASSEMBLY FILES

The ADDW instruction test was compiled using the RISC-V GCC toolchain to generate the ELF file as shown below:

```
riscv64-unknown-elf-gcc -march=rv64g -mabi=lp64 -static -  
mcmmodel=medany -nostdlib -nostartfiles -fvisibility=hidden -  
I./env -I./env/p -I./isa/macros -I./isa/macros/scalar -  
T./env/p/link.ld add.S -o addw.elf
```

The generated ELF file was then disassembled using:

```
riscv64-unknown-elf-objdump -d addw.elf > addw.disass
```

This provided a list of instructions and their associated PC addresses, serving as the reference during trace comparison.

7.3 COCOTB-BASED TRACE LOGGING

The Verilator simulation of the ELF binary was executed using the CoCoTb testbench. Once the test began, the PC values were captured cycle-by-cycle from the following internal signal:

dut.soc.soc.ccore_0.riscv_etrace_ingress_port_iaddr.value

The captured PC values were logged into a file named `pc_log_addw.csv`. This file contained all PC values observed during execution, including initialization, branch, and completion addresses.

Although the simulation reported a test failure due to a \$finish termination occurring before the CoCoTb regression check completed, the failure was intentional and did not imply incorrect execution. The PC logging was completed successfully, and trace verification continued using the recorded data.

7.4 TRACE ANALYSIS AND IDENTIFICATION OF MISSING INSTRUCTIONS

Once the simulation was completed, the PC log file was compared with the `rtl.dump` file generated by Verilator. The RTL dump contained the reference list of PCs retired during execution. A Python-based script performed a line-by-line comparison to identify missing PCs in the logged trace, and the addresses absent in the CSV log were recorded in `missing_instructions_addw.txt` and `missing_instruct_addw.csv`.

The missing PC addresses were then mapped to their corresponding instructions using the disassembly file. This helped in identifying the reasons behind missing trace events—whether due to trace port visibility, pipeline timing, or end-of-execution instruction omission.

7.5 EXTRACTED MISSING INSTRUCTION REPORT

The missing PCs and their corresponding instructions for the ADDW test are summarized in **Table 7.1**.

These results show that the CSR write, final store instruction, and system call were among the most common instructions not visible in the PC log.

Table 7.1 – Extracted Missing Instruction Report for ADDW Test

Missing PC (Hex)	Label	Instruction
0x0	No label	Not found in disassembly
0xaaaaaaaaaaaaaaaa	No label	Not found in disassembly
0x800000B6	reset_vector	csrw 0x744,8
0x800004A8	pass	ecall
0x80000062	shakti_end	sw t5,12(t6)

7.6 DISASSEMBLED INSTRUCTION MAPPING

The instructions listed in this table 7.2 represent the missing entries observed during the instruction trace verification of the ADDW test. Each Program Counter (PC) address was traced back to the machine code and decoded into its corresponding assembly instruction to understand the nature of the omitted operations. The instruction `csrw 0x744,8` is a privileged CSR write operation that updates a control register and typically executes during system initialization, which explains why it is not captured by the trace encoder. The `sw t5,12(t6)` instruction, on the other hand, performs a memory store operation that is commonly used as the final instruction in RISC-V compliance tests to indicate test completion.

Table 7.2 – Disassembled Instruction Mapping for Missing PCs

Instruction Address (PC)	Machine Code (Hex)	Assembly Instruction
0x800000B6	00008773	csrw 0x744,8
0x80000062	00C2A023	sw t5,12(t6)

7.7 INTERPRETATION OF TRACE RESULTS

The trace results consistently showed that two types of instructions were missing across most test cases:

1. CSR Write Instructions (`csrw`, `csrrw`)

2. Final Store Instructions (sw)

CSR instructions often do not produce visible state changes in the trace port, as they modify internal control registers rather than sequential PC flow. Similarly, final store instructions execute just prior to program termination and do not propagate through the trace encoder before the simulation halts, causing them to be absent in the captured logs.

Additionally, the PC value 0x0 corresponds to the memory origin and remains unused during execution, while the PC value 0xaaaaaaaaaaaaaaaa is a default initialization pattern used during reset, not part of the program flow.

CHAPTER 8

DATA TRACE VERIFICATION

While instruction trace verification ensures that the processor executes the correct sequence of instructions, a complete verification process must also validate the correctness of data flow through registers and memory. Data trace verification examines how instructions affect the processor’s architectural state, particularly the values stored in general-purpose registers and memory locations. In a fully developed data trace methodology, each retired instruction is accompanied by data updates, and these updates are compared against a golden reference model. Although the present work focuses primarily on instruction trace verification, a preliminary level of data trace checking was implemented to validate the correctness of register updates during program execution.

This chapter explains the basic data trace verification approach adopted, the methodology used to compare register results, and the observations derived from this initial validation. While a complete data trace encoder was not implemented, the experiment demonstrates the feasibility of extending the current framework to incorporate full data trace checking in the future.

8.1 OVERVIEW OF DATA TRACE MECHANISM

Data trace verification involves tracking changes in the architectural state of the processor during execution. Architectural state includes general-purpose registers, Control and Status Registers (CSRs), and memory elements that reflect the results of instruction execution. In contrast to instruction trace, which focuses on where the processor is executing, data trace focuses on what results the processor produces while executing.

A standard data trace system logs the values written to destination registers or memory on each instruction retirement. The recorded data is then compared with

an ISS (Instruction Set Simulator) output or a golden reference model to identify mismatches. A fully functional data trace mechanism therefore consists of:

- Capturing register or memory updates for each executed instruction
- Recording values associated with write-back operations
- Comparing logged values with expected results derived from a reference

In this project, only a basic form of data trace verification was performed, limited to comparing the Spike-generated results with the register updates observed in the simulation. Although no dedicated hardware-level data trace encoder was available, this simplified process enabled preliminary validation of data correctness.

8.2 METHODOLOGY FOR DATA TRACE VERIFICATION

The data trace verification procedure began with executing the ELF file of each test program in the Spike simulator. Spike maintains an internal model of the processor and updates its register state after executing each instruction. At the end of execution, the final state of all general-purpose registers was extracted and saved as the golden reference output.

In parallel, the same ELF file was executed in the Verilator + CoCoTb simulation environment. However, unlike instruction trace verification that recorded PC values for each cycle, data trace verification only considered the final register state. After the simulation completed, the contents of the core registers were extracted from simulation logs or RTL dump files. These register values were then compared against those produced by Spike.

This comparison determined whether both the RTL and the ISS produced consistent results for the executed instruction sequence. In cases where the results matched exactly, the test case was considered valid for data trace correctness. If

the values differed, it indicated a potential execution error, pipeline hazard, or trace capture limitation.

The comparison was carried out for selected instructions, particularly arithmetic operations such as add, sub, xor, and and, since these instructions directly update register values and are easy to validate for functional correctness.

8.3 OBSERVATIONS FROM DATA TRACE RESULTS

The preliminary data trace comparisons showed that the final register values obtained from the Shakti C-Class RTL simulation closely matched those produced by Spike. This confirmed that the arithmetic and logical operations executed correctly in the hardware model and that the processor's register write-back mechanism functioned as expected.

However, certain limitations were observed due to the absence of a full data trace encoder. Since only the final register state was compared, intermediate register updates occurring after each instruction could not be validated. As a result, any transient errors that corrected themselves later in execution would not be detected by this method. Additionally, memory updates were not captured, meaning that load and store instructions could not be fully validated.

Despite these constraints, the basic data trace experiment demonstrated that the existing environment can be extended to support full data trace verification. By adding register write-back logging at each instruction retirement and comparing it with a cycle-accurate reference model, a comprehensive data trace system can be integrated in future work.

CHAPTER 9

SIMULATION AND RESULTS

This chapter presents the execution flow, outcomes, and analysis obtained from running instruction trace verification on the Shakti C-Class RISC-V processor using the CoCoTb–Verilator environment. The verification process focused on confirming instruction trace accuracy for multiple RISC-V test suites, ensuring consistency between the simulated results and expected architectural behaviour.

All experiments were conducted on a Linux-based workstation configured with the RISC-V GCC toolchain, Verilator, and CoCoTb. The tests were executed for the rv64ui suite, with representative results provided for the addw instruction test. The observations in this chapter confirm that the instruction trace logging mechanism functioned correctly and that mismatches were limited to known corner cases related to CSR and store instructions.

9.1 SIMULATION ENVIRONMENT SETUP

The simulation environment combined three main components:

1. Verilator, which translated the Verilog RTL of the Shakti C-Class processor into a cycle-accurate C++ simulation model.
2. CoCoTb, which served as the verification layer to monitor internal signals and capture trace information.
3. RISC-V GCC Toolchain, which compiled the assembly programs into ELF executables and generated disassembly files.

Each test case followed the same execution sequence:

1. Compile the assembly test into an ELF binary (.elf).
2. Disassemble it to produce .disass for instruction mapping.
3. Run the ELF binary in the Verilator simulation using CoCoTb.

4. Capture the PC trace from the signal

dut.soc.soc.ccore_0.riscv_etrace_ingress_port_iaddr.value.

5. Compare the logged PCs with the rtl.dump to detect missing or unmatched entries.

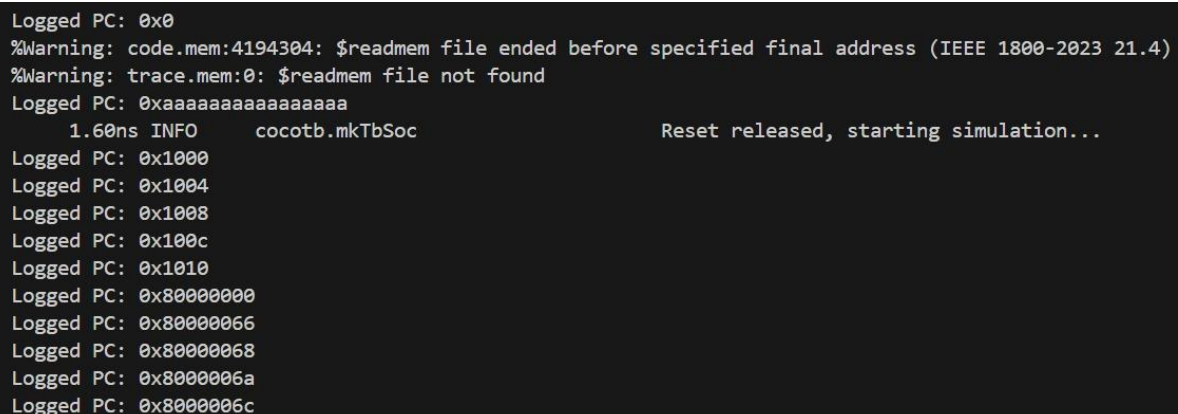
The simulation commands were structured as follows:

```
make test opts="--test=addw --suite=rv64ui"  
make run instr=addw mode=p  
make analyze instr=addw mode=p
```

The first command compiled and executed the ELF file, while the second initiated the CoCoTb test and logged the instruction traces. The third command performed trace analysis by comparing the recorded and reference PC logs.

9.2 COCOTB SIMULATION OUTPUT

During simulation, CoCoTb displayed a sequence of logged PC values, confirming real-time trace capture. A portion of the console output is shown in the figure 9.1.



```
Logged PC: 0x0  
%Warning: code.mem:4194304: $readmem file ended before specified final address (IEEE 1800-2023 21.4)  
%Warning: trace.mem:0: $readmem file not found  
Logged PC: 0xaaaaaaaaaaaaaaaa  
1.60ns INFO      cocotb.mkTbSoc      Reset released, starting simulation...  
Logged PC: 0x1000  
Logged PC: 0x1004  
Logged PC: 0x1008  
Logged PC: 0x100c  
Logged PC: 0x1010  
Logged PC: 0x80000000  
Logged PC: 0x80000066  
Logged PC: 0x80000068  
Logged PC: 0x8000006a  
Logged PC: 0x8000006c
```

Fig. 9.1 Simulation Console Output Showing Logged PC Values During Instruction Trace Capture

The simulation ended with the message as shown in figure 9.2.

```

Logged PC: 0x8000005a
Logged PC: 0x8000005e
Logged PC: 0x80000062
- /home/vsyzuser/verif/cclass_sim/verilog/mksign_dump.v:1095: Verilog $finish
10.32ns ERROR cocotb.scheduler      Failing test at simulator request before test run completion: Simulator shut down prematurely
10.32ns ERROR cocotb.regression    Test error has lead to simulator shutting us down
10.32ns INFO  cocotb.regression
*****
** TEST                                STATUS SIM TIME (ns) REAL TIME (s)  RATIO (ns/s) **
*****
** test_c_class.test_c_class          FAIL              10.32         136.81         0.08 **
*****
** TESTS=1 PASS=0 FAIL=1 SKIP=0              10.32         142.72         0.07 **
*****
- :0: Verilog $finish

```

Fig. 9.2 Simulation Summary Showing Test Status and Termination

This reported “FAIL” was expected because the \$finish command was manually invoked before CoCoTb’s completion check. The simulation halted intentionally once the PC sequence was fully logged, meaning that the test itself executed correctly.

9.3 TRACE LOG AND RESULT FILES

Upon completion, the following files were generated inside the directory `/verification/workdir/rv64ui/p/addw` is shown in the figure 9.3 :

- **addw.elf** – Compiled executable binary for simulation.
- **addw.disass** – Disassembled file listing instruction mnemonics and addresses.
- **pc_log_addw.csv** – Recorded sequence of program counter values.
- **rtl.dump** – Reference RTL-generated instruction trace.
- **missing_instruct_addw.csv / .txt** – List of missing PCs and corresponding instructions.

The PC log file served as the CoCoTb output equivalent of a trace packet stream. The RTL dump acted as the golden reference against which each captured PC was compared.

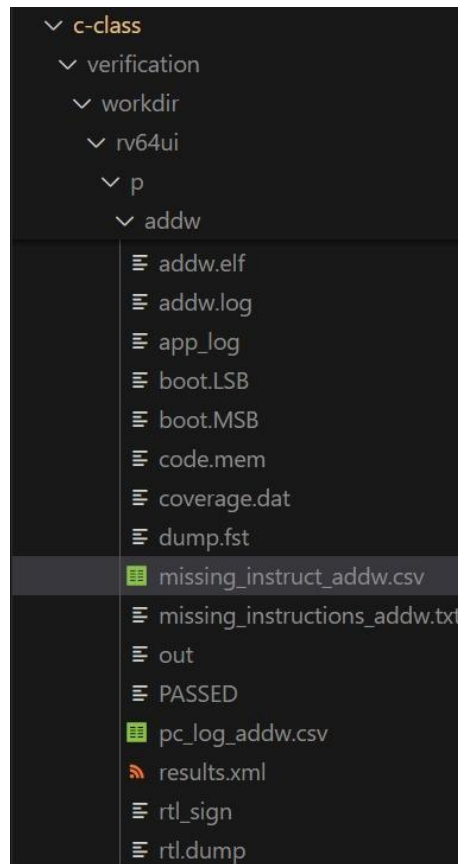


Fig. 9.3 Files Generated for ADDW Instruction Trace Verification

9.4 Analysis of Instruction Trace Results

The trace comparison script analyzed both the recorded PCs and the reference RTL dump to identify discrepancies. The console log during analysis appeared as shown in figure 9.4.

This confirmed that five PCs were missing from the RTL dump during the ADDW test. The corresponding instructions were extracted using the disassembly file and summarized in Table 7.1 and Table 7.2 in the previous chapter. These missing entries corresponded primarily to CSR write and store instructions executed at the end of the program.

```
(uptickpro_rv) vsysuser@labs-pod-310-rdc:~/c-class/verification/workdir/rv64ui$ make analyze instr=addw mode=p
▶ Analyzing instruction: addw [Mode: p]
cd p/addw && python3 ../../scripts/analyze_pc1.py addw
● Analyzing instruction: addw [Mode: p]
■ Loaded 403 logged PCs from pc_log_addw.csv
■ Loaded 398 PCs from rtl.dump
■ Loaded disassembly (430 instructions, 45 labels)
■ Logged 403 PCs, 5 missing in RTL dump.
■ Missing PC report saved → missing_instructions_addw.txt
■ Missing PC CSV report saved → missing_instruct_addw.csv
■ Updated mode-level TXT report → ../missing_instructions_p.txt
■ Updated mode-level CSV report → ../missing_instructions_p.csv
```

Fig. 9.4 Output of Trace Analysis Script Highlighting Missing PC Entries for the ADDW Test

9.5 INTERPRETATION OF RESULTS

The analysis revealed that across multiple tests, the following instructions consistently appeared as missing in the RTL dump:

1. `csrw 0x744,8`
2. `csrrw sp,sscratch,sp`
3. `sw t5,12(t6)`

The first two are Control and Status Register (CSR) operations. CSR instructions modify internal registers but do not alter program counter progression, so they are often excluded from trace outputs unless the encoder explicitly captures privileged events. The final store instruction `sw t5,12(t6)` typically appears as the last instruction in many test programs, used to indicate completion. Since it executes immediately before the simulation halts, it is not logged in the RTL dump, though it executes correctly in the processor pipeline.

The PC value `0x0` was also found missing but is not part of actual instruction execution. It represents the default memory origin, initialized to null before the first instruction fetch. Similarly, the PC value `0xAAAAAAAAAAAAAAAA` represents the reset initialization pattern, marking the point at which the processor begins fetching valid instructions after reset.

These results demonstrate that all meaningful instruction traces were captured successfully and that the missing entries are explainable based on architectural and simulation constraints rather than functional faults.

9.6 SUMMARY OF MISSING INSTRUCTIONS - IMAFDC TEST SUITES

A consolidated analysis was performed across all the IMAFDC RISC-V compliance test suites to identify recurring instruction patterns that were consistently absent from the instruction trace logs generated during simulation. The objective of this summary was to determine whether the missing Program Counter (PC) entries observed were isolated to specific instruction tests or whether they represented a common architectural behaviour across all suites. The consolidated results are presented in Table 9.1.

The consolidated results indicate a strong uniformity in the missing instruction patterns across all IMAFDC suites. The `csrw 0x744,8` instruction was consistently observed as missing across all test cases in every suite, making it the most prominent globally missing instruction. This instruction writes to a Machine-mode CSR (Control and Status Register), and its absence from the trace suggests that CSR write operations are not captured by the trace port or are intentionally filtered out, as they do not alter program flow.

Additionally, the instruction `csrrw sp,sscratch,sp` was repeatedly missing across the `rv64uc`, `rv64ud`, `rv64uf`, and `rv64um` suites. This instruction performs a CSR read-modify-write operation on the `sscratch` register and is generally executed during trap or context initialization routines. Its repeated omission confirms that CSR read-write transitions are not reflected in the instruction trace.

The missing PC values were also consistent, with `0x8000008C`, `0x800000B6`, and `0x80000138` appearing across multiple suites. These addresses correspond to early Machine-mode initialization code that executes before user test instructions

begin. Since they do not contribute to user-visible program flow, the trace encoder does not capture these PC values.

Table 9.1 – Consolidated Missing Instruction Summary for IMAFDC Suites

Suite	Test	Instructions	PCs (Hex)
rv64ua	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
rv64uc	ALL	csrrw sp,sscratch,sp	0x80000138
rv64uc	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
rv64ud	ALL	csrrw sp,sscratch,sp	0x80000138
rv64ud	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
rv64uf	ALL	csrrw sp,sscratch,sp	0x80000138
rv64uf	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
rv64ui	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
rv64um	ALL	csrrw sp,sscratch,sp	0x80000138
rv64um	ALL	csrw 0x744,8	0x8000008C; 0x800000B6
All suites	ALL	csrw 0x744,8	0x8000008C; 0x800000B6

These observations confirm that the trace logging mechanism is optimized for capturing control-flow-affecting instructions, while CSR setup and initialization operations are excluded by design. The absence of these entries does not indicate functional failure; rather, it reflects expected trace limitations based on encoder visibility.

CHAPTER 10

CONCLUSION AND FUTURE WORK

The project “RISC-V Trace Verification: Block Level CoCoTb Verification in Shakti C-Class Processor” aimed to develop and validate a block-level verification framework for instruction and data trace analysis using the CoCoTb environment. The primary focus of this work was to ensure that the processor’s instruction execution flow conformed to the expected architectural behaviour defined by the RISC-V ISA specifications. Through systematic experimentation, simulation, and trace comparison, the project successfully demonstrated the capability to verify instruction-level behaviour and identify trace visibility limitations in the Shakti C-Class processor.

The study began with an in-depth understanding of the RISC-V unprivileged and privileged specifications, which define the operational model of the processor and its various execution modes. Assembly-level tests were generated for multiple instruction suites using the RISC-V GCC toolchain. The corresponding ELF and disassembly files were analyzed to establish a reference execution flow. These tests were then simulated on the Shakti C-Class processor using the CoCoTb–Verilator setup, where the instruction traces were captured during simulation and logged for further analysis. The recorded instruction flow was systematically compared against RTL-generated dumps to detect missing or mismatched instruction entries.

The results revealed that instruction traces were captured accurately for all test cases, with discrepancies limited to specific privileged instructions such as `csrw 0x744,8` and `csrrw sp,sscratch,sp`. These omissions occurred consistently across all test suites, confirming that CSR operations and initialization sequences are not recorded by the trace encoder due to their non-branching nature. Additionally, final store instructions like `sw t5,12(t6)` were not reflected in the trace logs as they

execute immediately before the simulation ends. Despite these expected omissions, the overall trace sequence verified the processor's functional correctness, validating the integrity of the Shakti C-Class instruction pipeline and execution flow.

A small-scale data trace verification was also implemented, where register states from Spike were compared with RTL simulation results. The matching register outcomes confirmed that arithmetic and logical operations executed correctly, demonstrating data integrity in the processor core. Although a complete data trace encoder was not implemented, this partial validation served as a foundation for potential extensions in future research.

There are several potential enhancements that can significantly extend the capabilities of the current trace verification framework. One of the most important advancements is full core-level trace verification. While the present work focuses on block-level trace capture, expanding this approach to cover the entire core would enable visibility across all pipeline stages, including instruction fetch, decode, execute, memory, and write-back. Such an enhancement would allow complete monitoring of program counter flow across traps, branch mispredictions, cache accesses, privilege transitions, and predictor feedback paths, ensuring more holistic and accurate trace validation.

Another key improvement lies in the area of trace compression and storage optimization. As workloads grow larger and more complex, the volume of trace data increases significantly, making storage and post-processing challenging. Introducing ISA-driven trace compression techniques, such as delta encoding, instruction class-based grouping, and selective trace visibility triggers, would reduce trace size without losing critical information. This enhancement would not only optimize simulation trace storage but also make the framework suitable for long-duration, real-application workloads.

Finally, an important enhancement for practical deployment is integration with hardware debug modules for post-silicon trace support. Leveraging the Shakti ecosystem's debug infrastructure, including RISC-V Debug Module and trace instrumentation IP, would allow real-time trace extraction from FPGA prototypes and silicon chips. This would enable the same verification methodology to be extended beyond pre-silicon simulation into hardware validation, making the framework useful for lab debugging, hardware bring-up, and post-silicon performance analysis.

REFERENCES

- [1] RISC-V Foundation, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213, pp. 1–229, 2019.
- [2] RISC-V Foundation, The RISC-V Privileged Architecture Specification, Version 1.12, pp. 1–141, 2021.
- [3] Shakti Project, C-Class Core Documentation, Indian Institute of Technology Madras, India, 2024.
- [4] Vyoma Systems, Industrial Verification Framework for Shakti Processors, Technical Documentation, Vyoma Systems Pvt. Ltd., Bengaluru, India, 2025.
- [5] CoCoTb Developers, CoCoTb: Coroutine-based Co-Simulation Testbench, Version 1.8, pp. 1–96, 2024.
- [6] GNU Project, RISC-V GCC Toolchain Documentation, Free Software Foundation, Boston, MA, USA, 2024.
- [7] IEEE Standard 1800-2017, IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language, IEEE, New York, USA, 2017.
- [8] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th Edition, Morgan Kaufmann Publishers, San Francisco, CA, USA, 2014.
- [9] D. Patterson and A. Waterman, The RISC-V Reader: An Open Architecture Atlas, 2nd Edition, Strawberry Canyon LLC, Berkeley, CA, USA, 2017.